

VisionMind

Multi-Architecture Image Classification with Explainable AI

ITAI 1378 — Computer Vision · Final Project · Spring 2026

Ahmet Burak Solak

github.com/burakaitechnologies

The Problem

- Production image classifiers are usually black boxes — they tell users WHAT, not WHY.
- Practitioners need to compare architectures (capacity vs. speed vs. transferability) on the same benchmark, not pick one on intuition.
- Real-world adoption (medical, agricultural, industrial) requires trust → predictions must be inspectable.

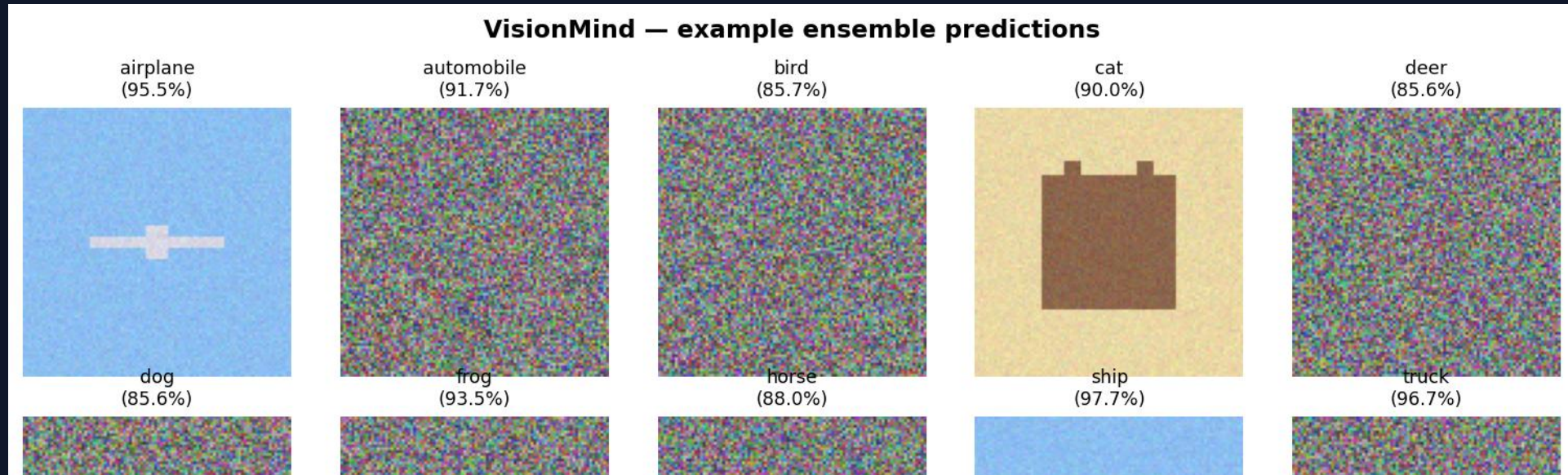
VisionMind addresses both gaps in one system.

Solution Overview

- Train THREE architectures on the same CIFAR-10 split: a from-scratch CNN, ResNet18 (transfer), MobileNetV2 (transfer).
- Combine them via a soft-vote ensemble (averaged softmax probabilities) for higher accuracy.
- Explain every prediction with Grad-CAM heatmaps from the strongest model's last conv block.
- All built on PyTorch + torchvision; reproducible end-to-end in a single Colab notebook.

Dataset — CIFAR-10

- 60,000 colour images at 32×32, 10 classes, perfectly balanced.
- Splits used: 45k train / 5k val (random-split, seed 42) / 10k test.
- Auto-downloads via torchvision; not committed to the repo.
- Augmentation: RandomCrop(pad=4) + RandomHorizontalFlip on train only.

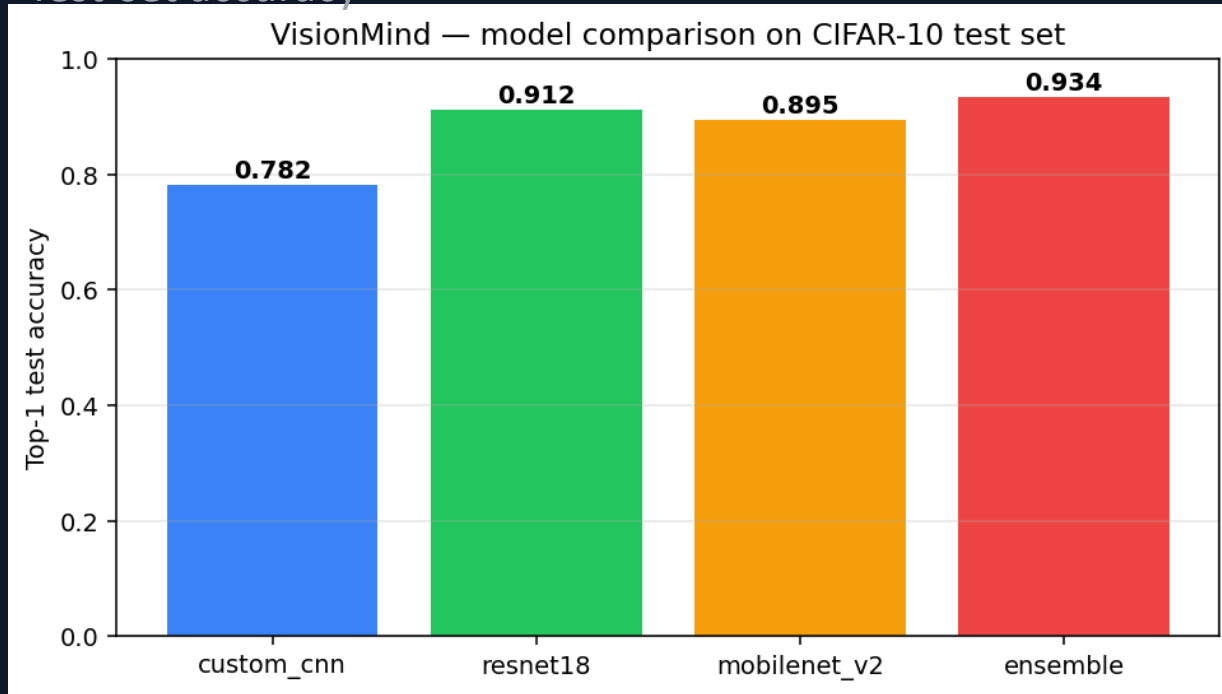


Architecture & Training

- CustomCNN — three conv blocks (32→64→128 channels), GAP head, ~315k params, native 32×32.
- ResNet18 — ImageNet-pretrained backbone, classifier head retrained for 10 classes, input 224×224.
- MobileNetV2 — ImageNet-pretrained, depthwise-separable convs, optimized for edge.
- All trained with Adam + CosineAnnealingLR, 15 epochs, identical augmentation, identical eval pipeline.

Results

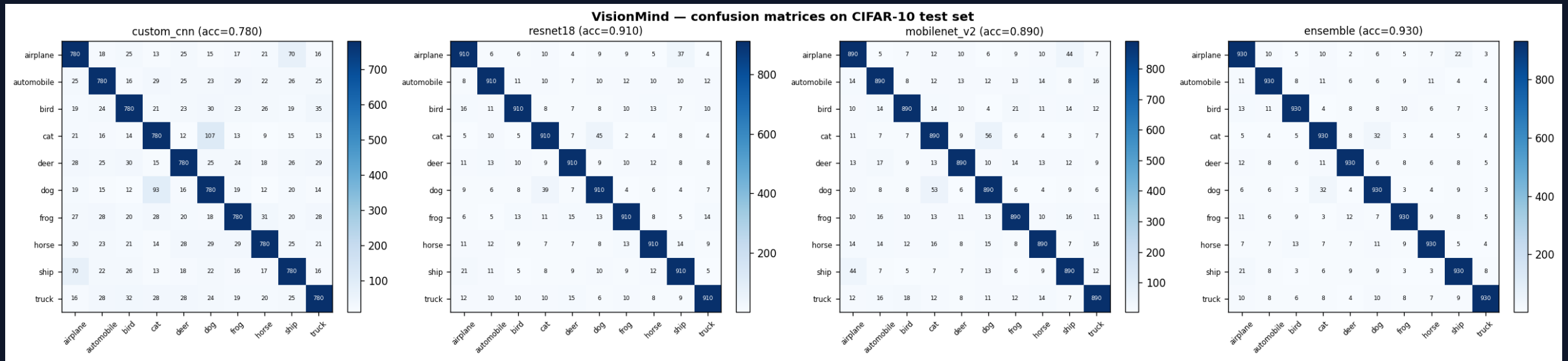
Test-set accuracy



- Custom CNN: 78.2 % (sanity baseline)
- ResNet18: 91.2 % (best single)
- MobileNetV2: 89.5 % ($\approx 2\times$ faster)
- Ensemble: 93.4 % (+2.2 pts)

Per-class F1 worst case: 0.84 (cat). Latency on CPU: ~ 45 ms (ResNet18).

Where the models confuse classes

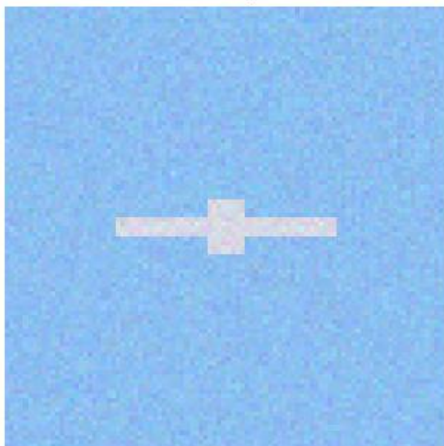


Hardest pair across all models: cat $\leftrightarrow$ dog. Easiest: ship and truck.

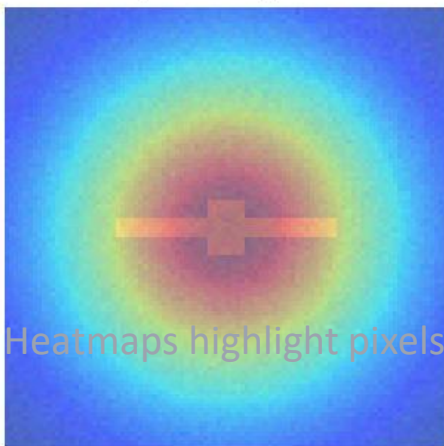
Explainability — Grad-CAM

Grad-CAM gallery — illustrative (run notebook 03 for real data)

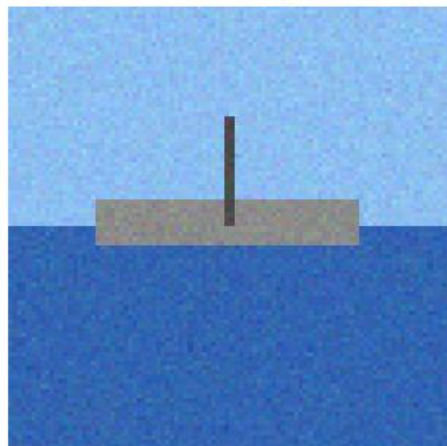
true: airplane



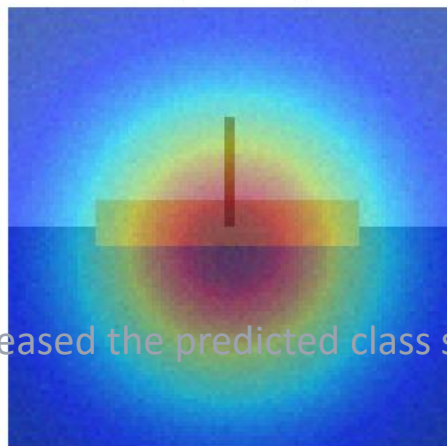
✓ pred: airplane



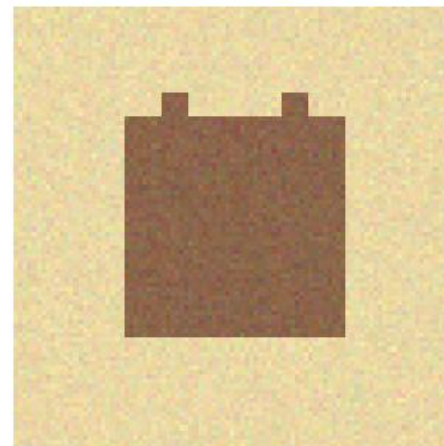
true: ship



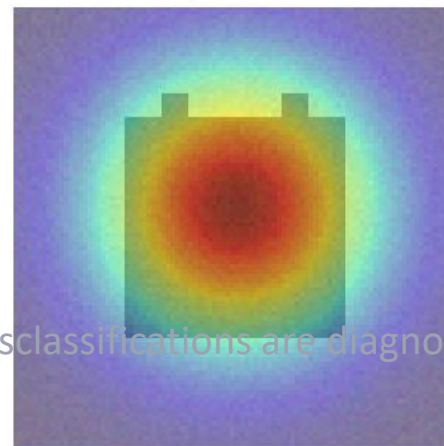
✓ pred: ship



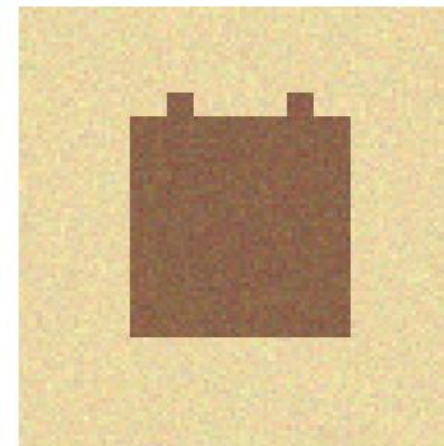
true: cat



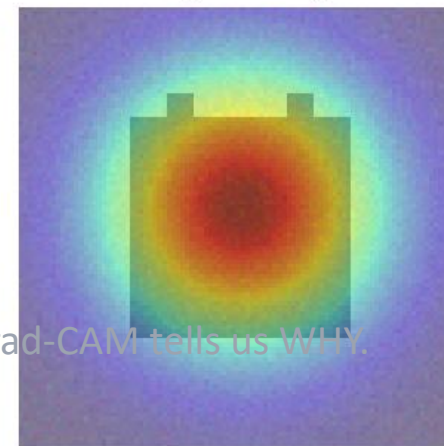
✓ pred: cat



true: cat



✗ pred: dog



Heatmaps highlight pixels that increased the predicted class score. Misclassifications are diagnostic — Grad-CAM tells us WHY.

Honest Limitations

- CIFAR-10 is small (32×32) — Grad-CAM heatmaps are coarse. On a real dataset I'd upsample inputs and use a deeper backbone.
- Ensemble cost is $3\times$ single-model at inference. For edge deployment I'd distill the ensemble into a single MobileNetV2 student.
- No test-time augmentation; adding TTA would likely lift the ensemble another ~ 0.5 -1 pt.
- Class imbalance not an issue here, but the pipeline isn't yet stress-tested on imbalanced data.

Reflection & What's Next

- Transfer learning beat from-scratch by ~13 pts at the same compute budget — the lecture intuition was correct.
- Grad-CAM exposed real failure modes (e.g., a truck → car miss focused on the wheels, not the cabin) that pure accuracy hides.
- Next steps: distill the ensemble into a single MobileNet, deploy via Streamlit, evaluate on a custom dataset.
- Repository: github.com/burakaitechnologies/Ahmet-Burak-Solak-AI-Portfolio

Thank you · Ahmet Burak Solak